

B 2.2.2 Arrays and Lists Python



B 2.2.2 Construct programs that apply arrays and Lists Python

This section explores how arrays and lists are used in Java and Python. We'll delve into one-dimensional and two-dimensional arrays, as well as ArrayLists. You'll learn how to manipulate these data structures, including adding, removing, and traversing elements.

Arrays

Arrays are static data structures that can store a set of elements of any type.

Python does not have built-in support for arrays, but Lists can be used instead in much the same way. However, Lists in Python are dynamic and not static like Arrays.

This page shows you how to use Lists as Arrays.

Elements are stored at locations that can be directly accessed through means of an index.

The diagram below shows how a set of integers can be stored in an array.

The array below has a size of 6 and for example, has the number 7445 stored at index 2.

Note that the first position in the array has an index 0.

Index	0	1	2	3	4	5
Data	3442	264	7445	345	2344	9334

2D Arrays

A two-dimensional array, also known as a matrix, is a data structure that stores elements in a grid-like format. 1 It's like a table with rows and columns, where each cell can hold a single value

2D arrays are useful for storing data that naturally falls into a grid-like structure, such as:

- Images (pixels)
- Game boards

- Spreadsheets
- Matrices
- Tables

To access an element in a 2D array, you need to specify its row and column indices. For example, to access the element at row 1, and column 2 in the above example, you would use.

```
myArray[1][2]
```

2D arrays are a powerful tool for storing and manipulating data in a structured way. They are used in many different programming languages and applications.

To declare new lists in Python we can do the following:

```
#Creation:
#arrayName = [value1, value2, ...]
#Example:
numbers = [1, 2, 3, 4, 5]
#Two-dimensional lists:
#arrayName = [[value1, value2], [value3, value4], ...]
#Example:
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

We will explore this in more detail below.

Lists

In Python, when you need to store a collection of items under a single variable name, a list is your go-to tool. Think of it as a flexible container that can hold an ordered sequence of items. These items can be of different data types (integers, strings, even other lists!).

Python offers four built-in data types for collections:

- **List:** Ordered and changeable (mutable), allows duplicate members.
- **Tuple:** Ordered and unchangeable (immutable), allows duplicate members.
- **Set:** Unordered and unindexed, no duplicate members.
- **Dictionary:** Unordered, changeable, and indexed, no duplicate members

For the syllabus, we will explore the **List** collection.

Creating a List

Lists are created in Python using square brackets.

List items are ordered, changeable, and allow duplicate values.

List items are indexed, the first item has an index [0], the second item has an index [1] and so on.

```
myList = ["Bob", "Alice", "Hopper", "Boule"]  
print(myList)
```

Adding to a List

To add an item to the end of a list we can use the **append()** method.

```
myList = ["Bob", "Alice", "Hopper", "Boule"]  
myList.append("Lovelace") #adds item to the end of the list  
print(myList)
```

Removing items from a List

To remove items from a list we can use the **remove()** method.

```
myList = ["Bob", "Alice", "Hopper", "Boule", "Lovelace"]  
myList.remove("Alice") #removes Alice from the list  
print(myList)
```

Traversing a list

You can loop through a List using a for loop.

```
myList = ["Bob", "Alice", "Hopper", "Boule", "Lovelace"]
for x in myList:
    print(x)
```

You can also loop through the List with a for loop using the index references.

```
myList = ["Bob", "Alice", "Hopper", "Boule", "Lovelace"]
for x in range(len(myList)):
    print(myList[x])
```

A while loop can also be used to loop through the List.

```
myList = ["Bob", "Alice", "Hopper", "Boule", "Lovelace"]
x=0
while x < len(myList):
    print(myList[x])
    x = x + 1
```

Two-dimensional Lists in Python

A two-dimensional List, also known as a matrix, is a data structure that stores elements in a grid-like format. It's like a table with rows and columns, where each cell can hold a single value

Two-dimensional Lists are useful for storing data that naturally falls into a grid-like structure, such as:

- Images (pixels)
- Game boards
- Spreadsheets
- Matrices
- Tables

To access an element in a two-dimensional List you need to specify its row and column indices. For example, to access the element at row 1, and column 2 in the above example, you would use.

```
myList[1][2]
```

Two-dimensional Lists are a powerful tool for storing and manipulating data in a structured way. They are used in many different programming languages and applications.

Creating a two-dimensional List

To declare a new two-dimensional List in Python we can do the following:

```
#Two-dimensional lists:  
listName = [[value1, value2], [value3, value4], ...]  
#Example:  
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Accessing values in a two-dimensional List

Think of a two-dimensional List like a table with rows and columns. To find a specific piece of data in this table, you need two pieces of information:

- The Row Number: This tells you which row the data is in.
- The Column Number: This tells you which column the data is in.

These row and column numbers are called indices (singular: index).

- **Two Indices to Access an Element:** To get to a specific element, you use both indices. For example, `myList[2][1]` refers to the element in the 3rd row (index 2) and 2nd column (index 1)
- **One Index to Access a Row:** If you only use one index, you're referring to an entire row within the 2D array. For example, `myList[2]` would give you all the elements in the 3rd row.

Example:

Imagine a grid of numbers represented as a 2D List:

```
grid = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
print(grid[1][2])  
#This would print 6  
print (grid [0])  
#this would print [1,2,3]
```

Traversing a two-dimensional List

To print out or traverse a 2D List we can use a nested loop to loop through the rows and columns.

```
T = [[1, 2, 3, 4], [5, 6, 7], [8, 9, 10, 11], [12,13,14,15]]  
for r in T:  
    for c in r:  
        print(c,end = " ")  
    print()
```

This would print out:

```
1 2 3 4  
  
5 6 7  
  
8 9 10 11  
  
12 13 14 15
```

There are a couple of things to take notice of in this code.

- In Python, the print() function has a default behaviour of adding a newline character at the end of each output. This means every time you use print(), the next output will start on a new line.
- The end parameter in print() allows you to change this default behaviour. By setting end = " ", you're telling the print() function to add a space instead of a new line at the end of each output.
- Also, notice there are not the same amount of elements in each row.

- 2D Lists do not need to have the same amount of elements on each row. This is known as a **jagged array** or a **ragged array**.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**